

C++ Roadmap for Implementing Matrix Operations Yourself

Goal: reach the point where you can implement the core linear-algebra operations used by ML/DL without depending on Eigen, NumPy, or another matrix library for the learning phase.

Important: this is not a roadmap to become a C++ specialist. C++ is being used as a tool to understand memory, data layout, algorithms, and what ML libraries are doing underneath the hood.

The dependency chain

1	Arrays & indexing	Understand contiguous data and manual indexing
2	Pointers & references	Understand addresses, mutation, and passing data
3	Dynamic memory / RAII	Understand ownership and heap allocation
4	std::vector	Use safe dynamic contiguous storage
5	Structs/classes	Build a Matrix abstraction
6	Nested loops & functions	Express mathematical operations
7	Memory layout	Understand row-major storage and cache behavior
8	Matrix operations	Implement add, multiply, transpose, etc.
9	Numerical algorithms	Implement dot products, norms, softmax, activations
10	Optimization	Improve allocations, locality, SIMD/threading later

Phase 0 — Reset your mental model

Before writing a Matrix class, understand what a matrix actually is in memory. A 3×4 matrix is not magically a mathematical object to the CPU: it can simply be 12 consecutive numbers plus information about its dimensions.

- **Learn:** stack vs heap, contiguous memory, addresses, sizeof, element size, indexing.
- **Exercise:** create an int array of 12 elements and manually interpret it as a 3×4 matrix.
- **Exercise:** calculate the address of element (row, col) using row-major indexing: $\text{index} = \text{row} * \text{cols} + \text{col}$.
- **Checkpoint:** you can explain why A[1][2] corresponds to one specific position in a flat array.

Phase 1 — The C++ memory tools you actually need

Concept	What you need to be able to do
Raw arrays	T arr[N], indexing, initialization, passing arrays to functions.
Pointers	T*, &, dereferencing, pointer arithmetic, nullptr.
References	T&, const T&, avoiding unnecessary copies.
const	const data, const parameters, const member functions.
Dynamic allocation	new/delete only to understand the mechanism; do not build your final Matrix around raw new/delete.
RAII	constructors/destructors and automatic resource ownership.
std::vector	size, data, resize, reserve, push_back, contiguous storage.
std::size_t	Use it naturally for dimensions and indexes.

Mini-projects before Matrix

- **P1 — Array statistics:** min, max, mean, variance using only loops.

- **P2 — Dynamic array:** implement a tiny `DynamicArray` class using heap allocation.
- **P3 — Vector practice:** implement the same algorithms with `std::vector`.
- **P4 — Dot product:** `dot(a,b)`, with dimension checking.
- **P5 — 2D grid:** store and print a 2D image using a flat vector.

Phase 2 — Build your first Matrix class

Do NOT start with a sophisticated template library. Start with a simple educational Matrix. Once it works, generalize it.

Recommended internal representation

```
class Matrix {
    size_t rows_;
    size_t cols_;
    std::vector<double> data_;
};
```

- Constructor: `Matrix(rows, cols)`.
- Element access: `operator()(row, col)`.
- Bounds checking in a separate checked accessor or during debugging.
- `rows()`, `cols()`, `size()`.
- Fill with a value; create zeros and ones.
- Print the matrix neatly.
- Random initialization for later ML experiments.
- Understand exactly why `data_[row * cols_ + col]` accesses `(row, col)`.

Phase 3 — Implement matrix operations in this order

Order	Implementation	Difficulty
1. Scalar operations	$A + c$, $A - c$, $A * c$	Easy
2. Matrix addition/subtraction	$C(i,j) = A(i,j) \pm B(i,j)$	Easy
3. Element-wise multiplication	Hadamard product	Easy
4. Transpose	$C(j,i) = A(i,j)$	Easy
5. Row/column extraction	Useful for debugging and ML	Easy
6. Dot product	vector $\cdot$ vector	Easy
7. Matrix-vector multiplication	$y = A \cdot x$	Medium
8. Matrix-matrix multiplication	$C = A \cdot B$	Medium
9. Sum / mean / max	Reduction operations	Medium
10. Norms	L1/L2/Frobenius	Medium
11. Broadcasting-like operations	Add vector to rows/columns	Medium
12. Element-wise functions	exp, log, sqrt, tanh	Medium

The matrix multiplication milestone

For A with shape (m,n) and B with shape (n,p) , C has shape (m,p) . The core algorithm is three loops:

```
for (i = 0; i < m; ++i)
  for (j = 0; j < p; ++j)
    for (k = 0; k < n; ++k)
      C(i, j) += A(i, k) * B(k, j);
```

Do not merely copy this. Derive it on paper from the mathematical definition, then implement it yourself. Afterwards, inspect the memory-access pattern and understand why loop ordering affects performance.

Phase 4 — Turn the Matrix into an ML learning tool

ML concept	Implement with your Matrix/vector code
Linear layer	$Y = XW + b$
Neuron	$\text{dot}(x, w) + b$
ReLU	$\max(0, x)$ element-wise
Sigmoid	$1 / (1 + \exp(-x))$
Softmax	stable exponentials + normalization
MSE	$\text{mean}((\text{prediction} - \text{target})^2)$
Cross-entropy	log probabilities + reduction
Gradient descent	$\text{parameter} -= \text{learning_rate} * \text{gradient}$

Phase 5 — Your first from-scratch neural network

- **Project 1:** linear regression using your Matrix/vector implementation.
- **Project 2:** logistic regression on a tiny binary dataset.
- **Project 3:** 2-layer neural network on XOR.
- **Project 4:** small fully-connected network on MNIST — only after the previous projects work.

- **Project 5:** compare your implementation against NumPy/PyTorch outputs on identical inputs.

Phase 6 — Optimization: only after correctness

Your goal is not to write the fastest matrix library immediately. First make the mathematics correct. Then learn why high-performance ML libraries are fast.

- **Memory locality:** cache lines, sequential access, row-major layout.
- **Allocation:** avoid creating temporary matrices inside hot loops.
- **Move semantics:** understand copies vs moves for large Matrix objects.
- **const references:** avoid unnecessary copies when passing matrices.
- **Loop ordering:** compare i-j-k, i-k-j, etc. for matrix multiplication.
- **Blocking/tiling:** improve cache reuse.
- **SIMD:** understand vectorized arithmetic conceptually, then experiment.
- **Multithreading:** parallelize independent rows/blocks.
- **Benchmarking:** measure before and after optimization.

What you do NOT need yet

- Do not learn every corner of modern C++ before touching matrices.
- Do not build a giant Eigen replacement.
- Do not start with templates, expression templates, allocators, metaprogramming, or advanced concepts.
- Do not jump into CUDA before you understand CPU memory and matrix multiplication.
- Do not optimize code that you cannot yet explain mathematically.
- Do not abandon Python/ML: use Python/PyTorch/NumPy as the reference implementation and C++ as the under-the-hood laboratory.

Your concrete progression

Stage	You should be able to...	Then move on
A	Manipulate arrays, pointers, references, vectors	Dynamic array
B	Understand contiguous memory and indexing	Matrix class
C	Build Matrix(rows, cols) with vector<double>	Basic operations
D	Implement add, transpose, dot, matvec	Matmul
E	Implement matrix multiplication from the definition	ML math
F	Implement activations/losses/gradient descent	Neural network
G	Train a tiny NN from scratch	Optimization
H	Profile/cache-optimize your implementation	Advanced systems

The rule for your learning plan

Learn just enough C++ → implement one primitive → test it → use it in an ML problem → inspect/optimize it.

For your goal, matrix operations are not a random side quest. They are the bridge between your current Python/ML work and understanding what numerical libraries, neural-network frameworks, and optimized AI infrastructure are doing underneath the hood.

Suggested stopping point for the first cycle: a correct Matrix class + matrix multiplication + linear layer + activation functions + loss + gradient descent + a tiny neural network. Only then begin serious optimization.